# LAPORAN PRAKTIKUM MACHINE LEARNING
# DECISION_TREES




Oleh:

Nama : Dyelvio Wijaya

NIM : 211344012

Tanggal Praktikum : 27 Oktober 2023

Tanggal Pengumpulan Laporan : 31 Oktober 2023

**PRODI D4 TEKNIK TELEKOMUNIKASI - TEKNIK ELEKTRO**
**POLITEKNIK NEGERI BANDUNG**
**2023/2024**

A. Program Decission Tree

Import Library

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
import sklearn.tree as tree
```

Dowload Data

```
!wget 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-ML0101EN-SkillsNetwork/labs/Module%203/data/drug200.csv'
```

```
--2023-10-29 04:11:00--  https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDeveloperSkillsNetwork-ML0101EN-SkillsNetwork/labs/Module%203/data/drug200.csv
Resolving cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud)... 198.23.119.245
Connecting to cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud (cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud)|198.23.119.245|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 5827 (5.7K) [text/csv]
Saving to: ‘drug200.csv’

drug200.csv         100%[===================>]   5.69K  --.-KB/s    in 0s

2023-10-29 04:11:00 (1023 MB/s) - ‘drug200.csv’ saved [5827/5827]
```

Load Data dari file CSV

```
#koma adalah karakter yang harus digunakan sebagai pemisah atau delimiter saat memproses atau membaca data.
my_data = pd.read_csv("drug200.csv", delimiter=",")
my_data[0:5]
```

| | Age | Sex | BP | Cholesterol | Na_to_K | Drug |
|---|---|---|---|---|---|---|
| 0 | 23 | F | HIGH | HIGH | 25.355 | drugY |
| 1 | 47 | M | LOW | HIGH | 13.093 | drugC |
| 2 | 47 | M | LOW | HIGH | 10.114 | drugC |
| 3 | 28 | F | NORMAL | HIGH | 7.798 | drugX |
| 4 | 61 | F | LOW | HIGH | 18.043 | drugY |

Practice untuk mencari size dari data

```
# write your code here
my_data.shape
```

```
(200, 6)
```

Pre-processing

```
X = my_data[['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K']].values
X[0:5]
```

```
array([[23, 'F', 'HIGH', 'HIGH', 25.355],
       [47, 'M', 'LOW', 'HIGH', 13.093],
       [47, 'M', 'LOW', 'HIGH', 10.114],
       [28, 'F', 'NORMAL', 'HIGH', 7.798],
       [61, 'F', 'LOW', 'HIGH', 18.043]], dtype=object)
```

```
from sklearn import preprocessing
#Fungsi utama LabelEncoder adalah mengambil serangkaian label kelas yang bersifat
#kualitatif (misalnya, "Merah", "Biru", "Hijau") dan mengubahnya menjadi label-label
#yang bersifat kuantitatif (misalnya, 0, 1, 2)
le_sex = preprocessing.LabelEncoder()
#memahami bahwa 'F' adalah label kelas dengan kode 0 dan 'M' adalah label kelas dengan kode 1
le_sex.fit(['F','M'])
#le_sex.transform(X[:, 1]) adalah pemanggilan metode transform dari objek le_sex
#Metode ini digunakan untuk mengkodekan data dalam bentuk numerik berdasarkan
#informasi yang telah dipelajari oleh LabelEncoder saat pemanggilan fit sebelumnya
#Hasilnya, kolom kedua dari X akan berisi kode-kode numerik yang merepresentasikan kategori-kategori tersebut
X[:,1] = le_sex.transform(X[:,1])


le_BP = preprocessing.LabelEncoder()
#untuk mengkodekan label kelas dari data kategori (kualitatif) dalam kolom tertentu menjadi bentuk numerik
le_BP.fit([ 'LOW', 'NORMAL', 'HIGH'])
#digunakan untuk mengkodekan (encode) kolom ketiga dari array X menggunakan objek LabelEncoder yang disebut le_BP
X[:,2] = le_BP.transform(X[:,2])


le_Chol = preprocessing.LabelEncoder()
le_Chol.fit([ 'NORMAL', 'HIGH'])
#digunakan untuk mengkodekan (encode) kolom keempat dari array X menggunakan objek LabelEncoder yang disebut le_Chol
X[:,3] = le_Chol.transform(X[:,3])

X[0:5]
# M = 1, F = 0
# HIGH = 0, LOW = 1, Normal = 2
# NORMAL = 1, HIGH = 0
```

```
array([[23, 0, 0, 0, 25.355],
       [47, 1, 1, 0, 13.093],
       [47, 1, 1, 0, 10.114],
       [28, 0, 2, 0, 7.798],
       [61, 0, 1, 0, 18.043]], dtype=object)
```

Data Output untuk target variable

```
y = my_data["Drug"]
y[0:5]
```

```
0    drugY
1    drugC
2    drugC
3    drugX
4    drugY
Name: Drug, dtype: object
```

Setting Up the Decision Tree

```
from sklearn.model_selection import train_test_split
```

```
#Pembagian Trainset 70% dan Testset 30%
X_trainset, X_testset, y_trainset, y_testset = train_test_split(X, y, test_size=0.3, random_state=3)
```

```
print ('Train set:', X_trainset.shape,  y_trainset.shape)
print('Shape of X training set {}'.format(X_trainset.shape),'&',' Size of Y training set {}'.format(y_trainset.shape))
```

```
Train set: (140, 5) (140,)
Shape of X training set (140, 5) &  Size of Y training set (140,)
```

```
print ('Test set:', X_testset.shape,  y_testset.shape)
print('Shape of X training set {}'.format(X_testset.shape),'&',' Size of Y training set {}'.format(y_testset.shape))
```

```
Test set: (60, 5) (60,)
Shape of X training set (60, 5) &  Size of Y training set (60,)
```

Modeling

```
# untuk membuat sebuah objek model Decision Tree (Pohon Keputusan) dalam machine learning
# max_depth=4: Ini adalah parameter yang menentukan kedalaman maksimum decission tree
drugTree = DecisionTreeClassifier(criterion="entropy", max_depth = 4)
drugTree # it shows the default parameters
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=4)
```

```
drugTree.fit(X_trainset,y_trainset)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(criterion='entropy', max_depth=4)
```

Prediction

```
#untuk melakukan prediksi label kelas pada data uji (X_testset)
#menggunakan model Decision Tree yang telah dibuat sebelumnya (drugTree)
predTree = drugTree.predict(X_testset)
```

```
# Menampilkan nilai hasil prediksi
print (predTree [0:5])
# menampilkan Nilai y_testset
print (y_testset [0:5])
```

```
['drugY' 'drugX' 'drugX' 'drugX' 'drugX']
40     drugY
51     drugX
139    drugX
197    drugX
170    drugX
Name: Drug, dtype: object
```

Evaluation

```
from sklearn import metrics
import matplotlib.pyplot as plt
# untuk menghitung akurasi dari model Decision Tree pada data uji (X_testset dan y_testset) berdasarkan hasil prediksi model (predTree)
print("DecisionTrees's Accuracy: ", metrics.accuracy_score(y_testset, predTree))
```

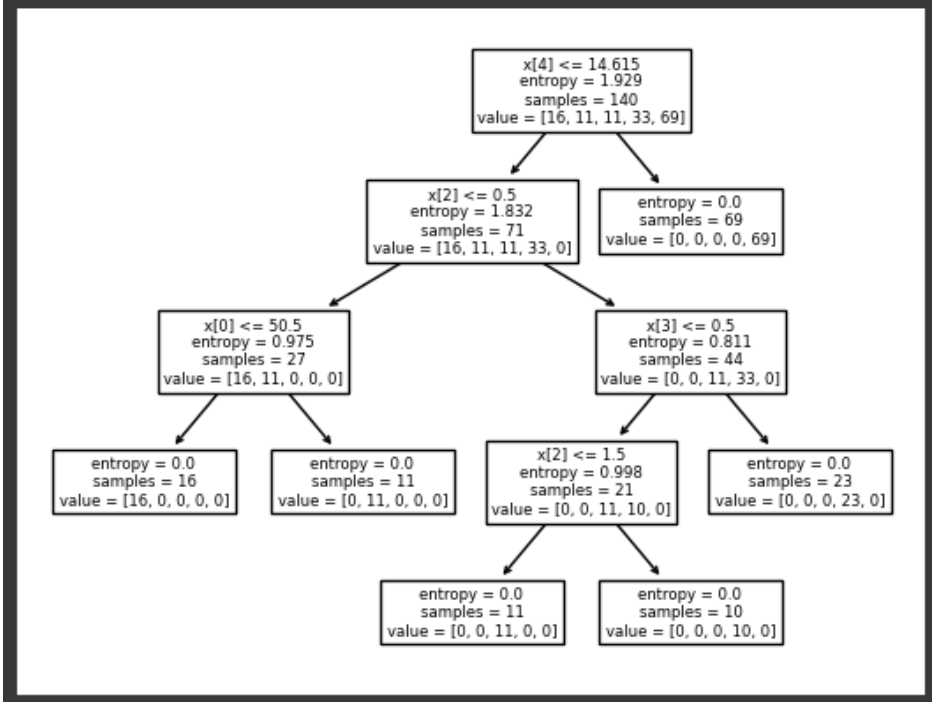
```
DecisionTrees's Accuracy:  0.9833333333333333
```

Visualization

```
# Notice: You might need to uncomment and install the pydotplus and graphviz libraries if you have not installed these before
!conda install -c conda-forge pydotplus -y
!conda install -c conda-forge python-graphviz -y
```

```
/bin/bash: line 1: conda: command not found
/bin/bash: line 1: conda: command not found
```

```
#untuk membuat visualisasi grafis dari model Decision Tree (drugTree) yang telah dibuat
tree.plot_tree(drugTree)
plt.show()
```




B. Analisa

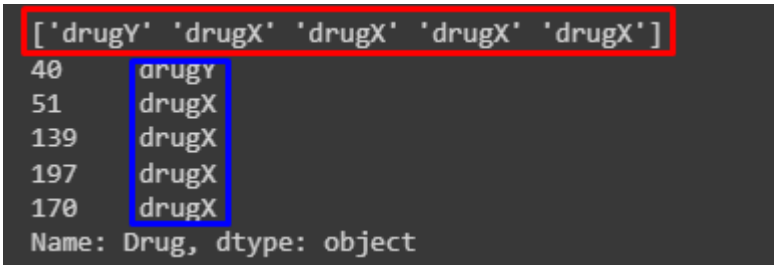
Decision Tree (pohon keputusan) adalah model prediktif yang digunakan dalam machine learning dan statistik. Ini adalah salah satu algoritma yang paling umum digunakan untuk tugas klasifikasi dan regresi. Decision Tree mengambil bentuk pohon dengan node-node yang mewakili keputusan atau pemisahan berdasarkan fitur-fitur dari data input. Dalam pembuatan model decission tree dibutuhkan libraray numpy, pandas, sklearn.tree yang mengambil DecisionTreeClassifier, kemudian sklearn.model_selection dengan mengambil train_test_split, sklearn mengambil metics dan preprocessing, kemudian library sklearn.tree. Dalam pembuatan model decission tree diperlukan proses label encoder. Proses Label Encoding digunakan saat ada fitur kategori (data kualitatif atau data kategori) yang perlu diubah menjadi data numerik. Ini diperlukan karena algoritma Decision Tree dan sebagian besar algoritma machine learning bekerja dengan baik pada data numerik.

Lalu untuk alasan digunakannya sebuah label encoding dalam proses pembuatan model decision tree dikarenakan keperluan data numerik, untuk perhitungan entropi, penyusunan aturan pemisahan, kemudahan interpretasi, dan kompabilitas dengan library dikarenakan banyak library dan implementasi Decision Tree dalam bahasa pemrograman seperti scikit-learn di Python, membutuhkan data numerik. Namun perlu diketahui juga bahwa Label Encoding harus digunakan dengan hati-hati. Beberapa algoritma Decision Tree, seperti ID3, yang

mengukur ketidakpastian dengan entropi, mungkin menghasilkan pohon yang cenderung overfitting jika Label Encoding digunakan pada data yang memiliki banyak kategori unik. Dalam kasus seperti itu, penggunaan teknik lain seperti One-Hot Encoding atau pengkodean kategori yang lebih canggih dapat lebih baik dalam mengatasi data kategori.

Dalam praktik kali ini proses label encoder terdapat pada kolom Sex, BP, dan Cholestrol. Dimana untuk kolom sex M akan sama dengan 1 dan untuk F akan sama dengan 0, kemudian dalam kolom BP, untuk LOW akan bernilai numerik 1, HIGH bernilai numerik 0, dan Normal akan bernilai numerik 2, lalu untuk kolom Cholestrol untuk NORMAL akan bernilai numerik 1, dan HIGH akan bernilai numerik 0. Kemudian dalam pembagian dataset , sama seperti praktik sebelumnya dalam proses pembuatan model kita perlu membagi terlebih dahulu data menjadi dua bagian yaitu untuk train data dan test data. Train data perlu memiliki data set yang lebih banyak atau lebih baik diatas 60% dataset. Namun untuk Testset hanya diperlukan sekitar 10 – 40% dikarenakan perlu juga adanya pengetesan untuk menilai apakah model yang kita buat sudah baik atau tidaknya. Layaknya manusia yang perlu banyak belajar agar saat nantinya saat test akan menghasilkan nilai yang bagus, begitu juga dengan machine learning pun sama seperti manusia sedang mempersiapkan test. Dalam praktik kali ini Train Set menggunakan 70% dan Test set 30%.

Dalam proses modeling ini dalam percobaan untuk kedalaman nya maksimal adalah 4, dan saat dalam percobaan untuk prediksi dapat terlihat dalam hasil prediksi dengan test set didapatkan sama, dapat dilihat dengan gambari dibawa ini:

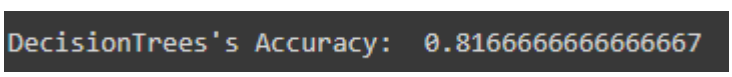


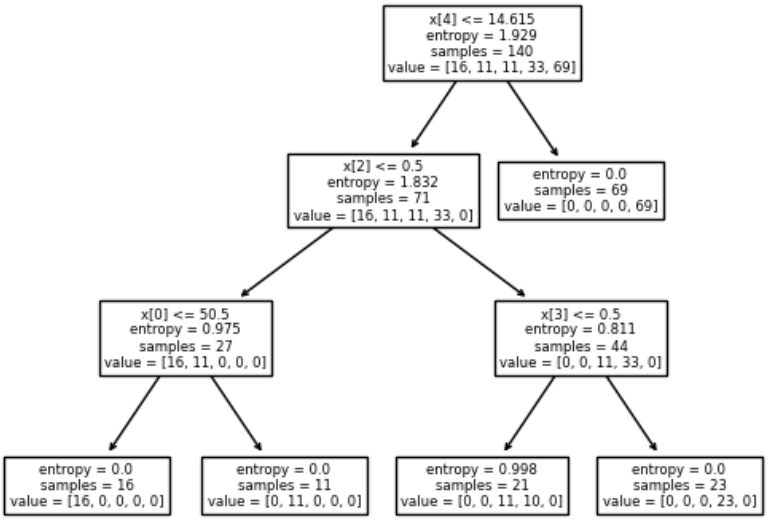

Keterangan:

Kotak Merah Hasil Prediksi

Kotak Biru Test Set

Sehingga dapat terlihat bahwa hasil model yang telah dibuat memiliki nilai akurasi yang tinggi, dikarenakan memiliki kesamaan yang sama dengan test set. Dapat dibuktikan juga dalam proses evaluasi model didapatkan bahwa nilai akurasinya yaitu 0.983 yang hampir mendekati 1, karena jika model sangat cocok maka nilai akurasi akan 1, dan jika masih tidak ada kesamaan sama sekali akan bernilai 0. Kemudian untuk melihat nilai entropy dapat digunakan dengan memanggil fungsi tree.plot_tree() dan plt.show(). Maka nantinya akan terlihat entropy dari model berapa dalam setiap subset, dimana entropy sendiri adalah salah satu metrik yang digunakan untuk mengukur ketidakpastian atau kebingungan dalam data. Semakin rendah entropy pada node, semakin baik model dalam melakukan pemisahan kelas target. Lalu apabila max depth kita rubah menjadi 3, maka nantinya nilai akurasinya akan menjadi lebih rendah yaitu 0.8167.






Kemudian dalam gambar decission tree untuk max depth = 3, pun masih ada nilai entropy yang belum 0, sehingga dapat memengaruhi akurasi pada model, berbeda dengan max depth yang digunakan dalam percobaan utama yaitu 4, dan dalam decision tree nya pun tidak ada entropy yang masih tersisa(entropy dalam ujung node = 0).